

Machine Learning for Business

GTSC2143

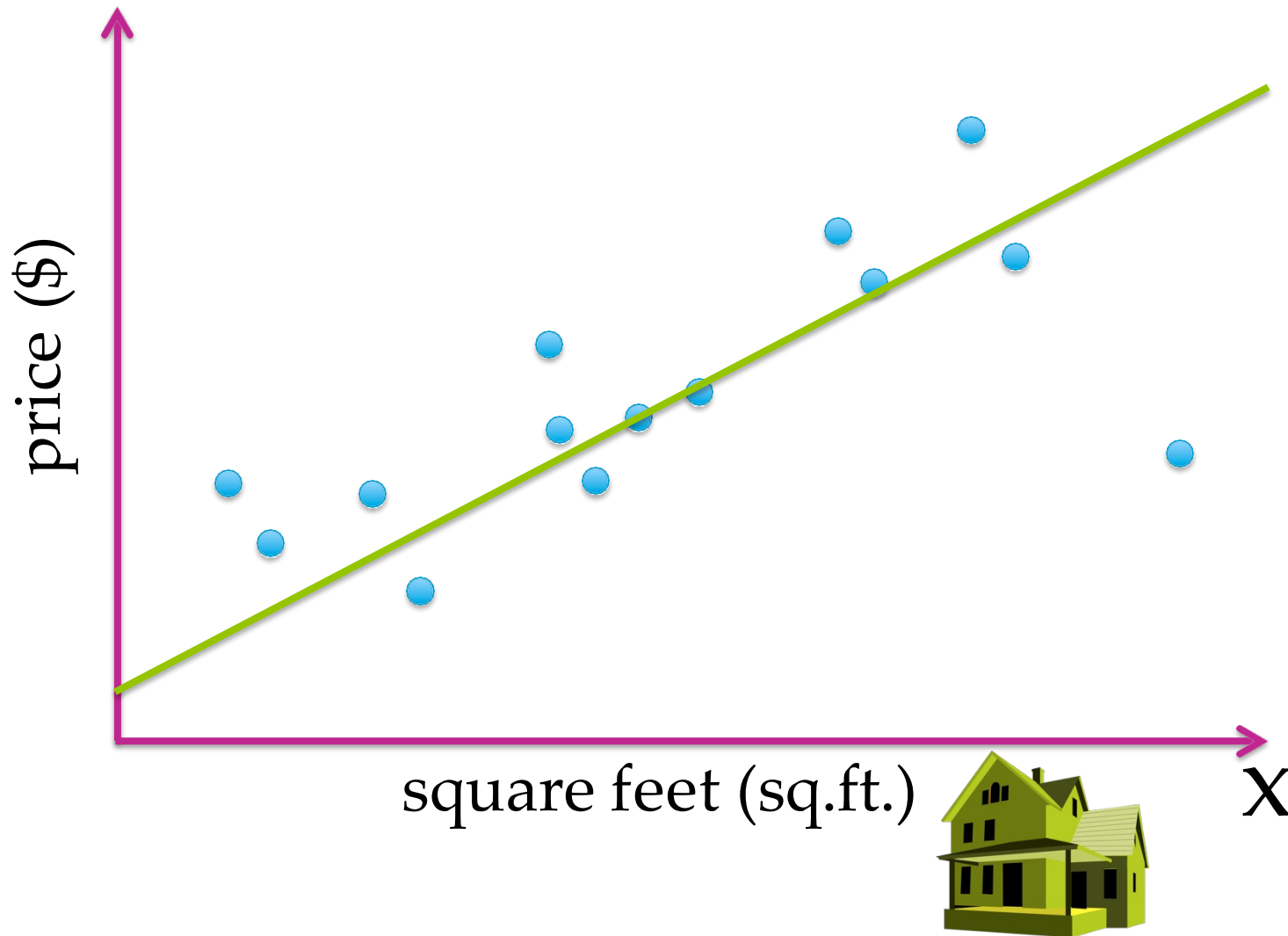
Lecture 5 Regression II

Instructor: Dr. Lily Bei Luo

AY2025-2026 Semester 1

Quick Review: Linear Regression

What we learned: predicting house prices

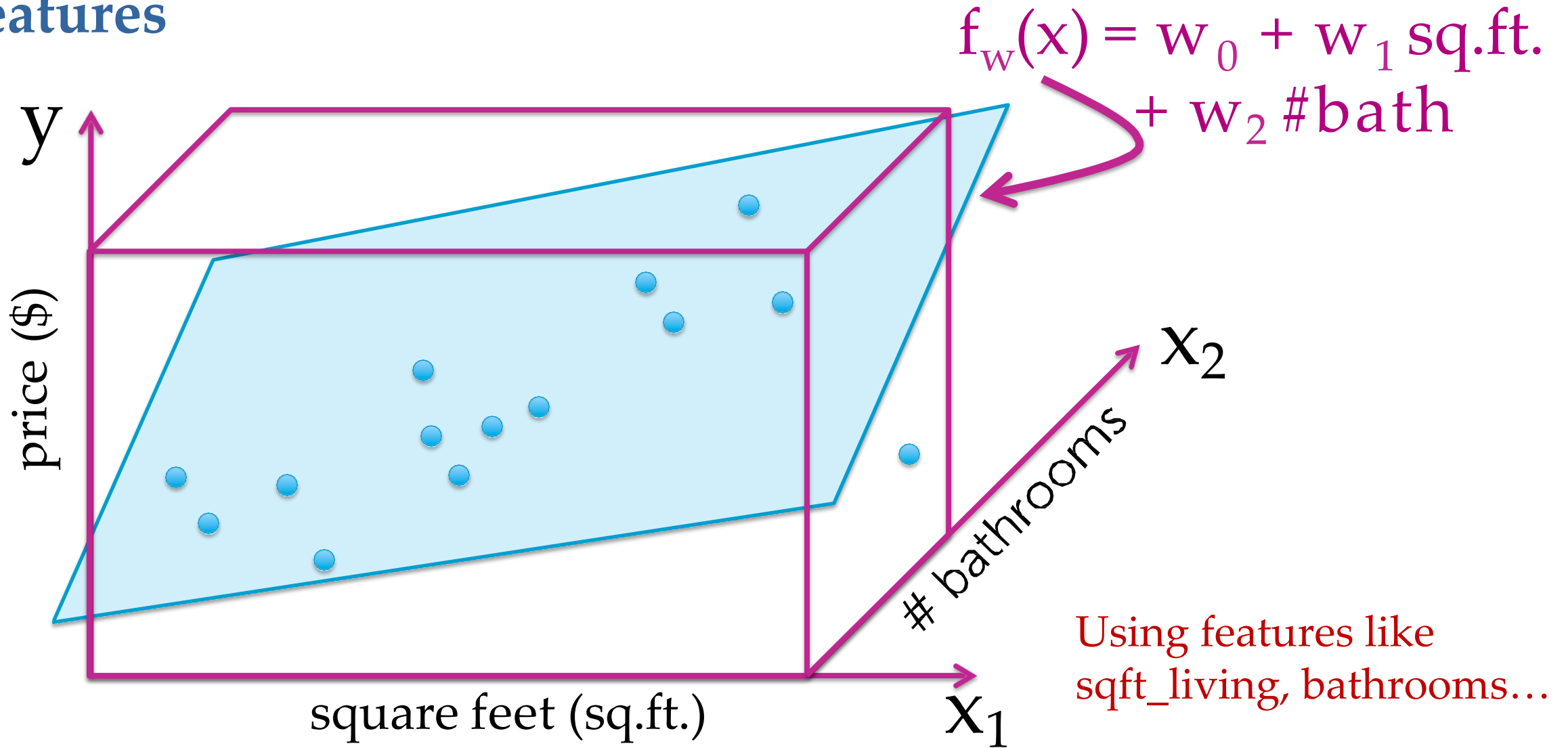


Predicting continuous house prices (\$)

How much is my house worth?

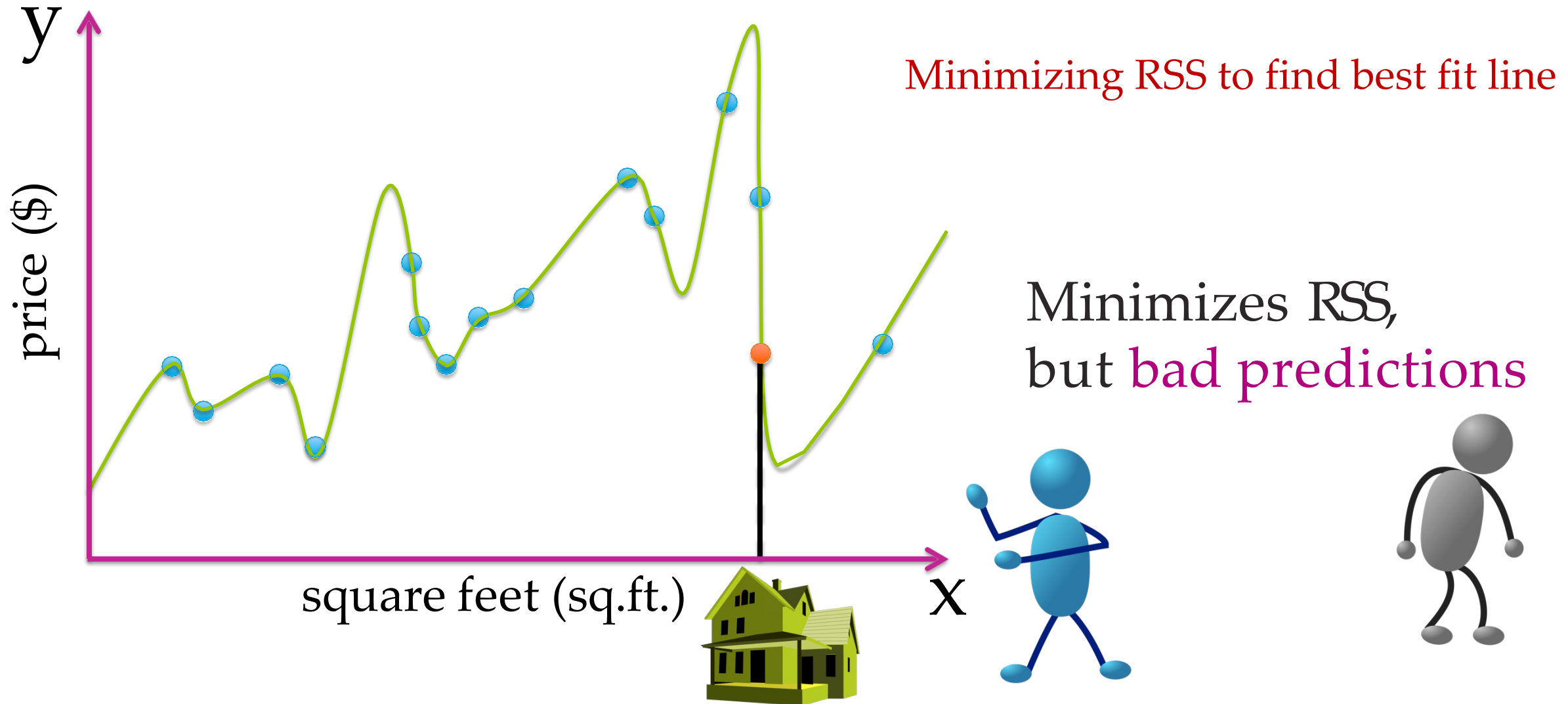
What we learned: predicting house prices

Features



What we learned: predicting house prices

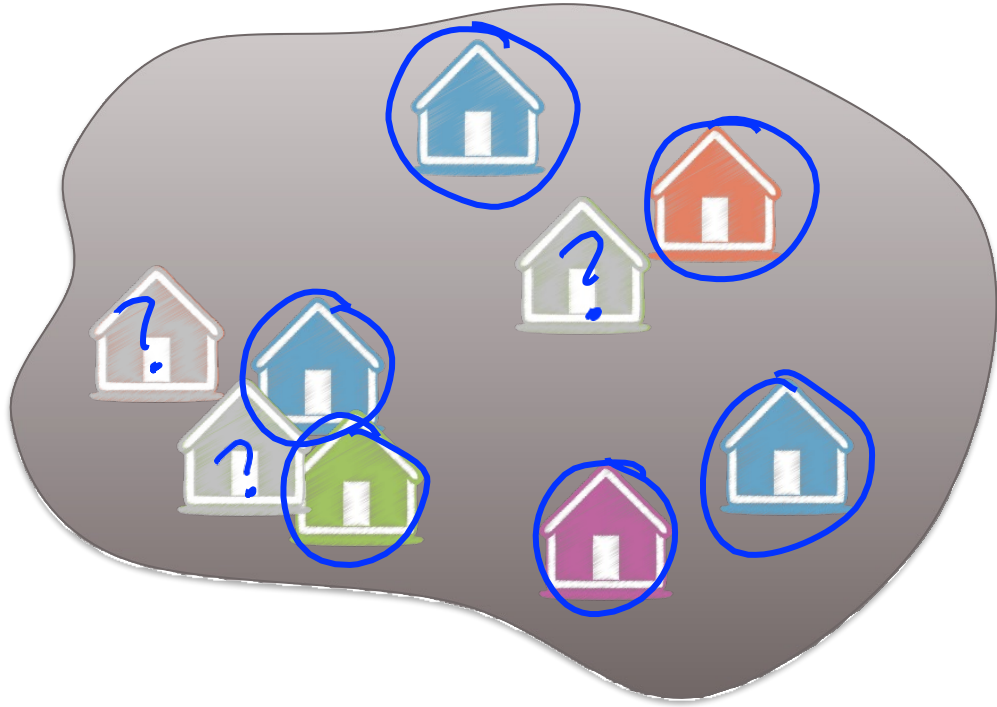
Overfitting



What we learned: predicting house prices

Training/Test Split

Training/test split to avoid overfitting



- Want good predictions, but can't observe future
- Simulate predictions
 1. Remove some houses
 2. Fit model on remaining
 3. Predict heldout houses

Terminology: – training set



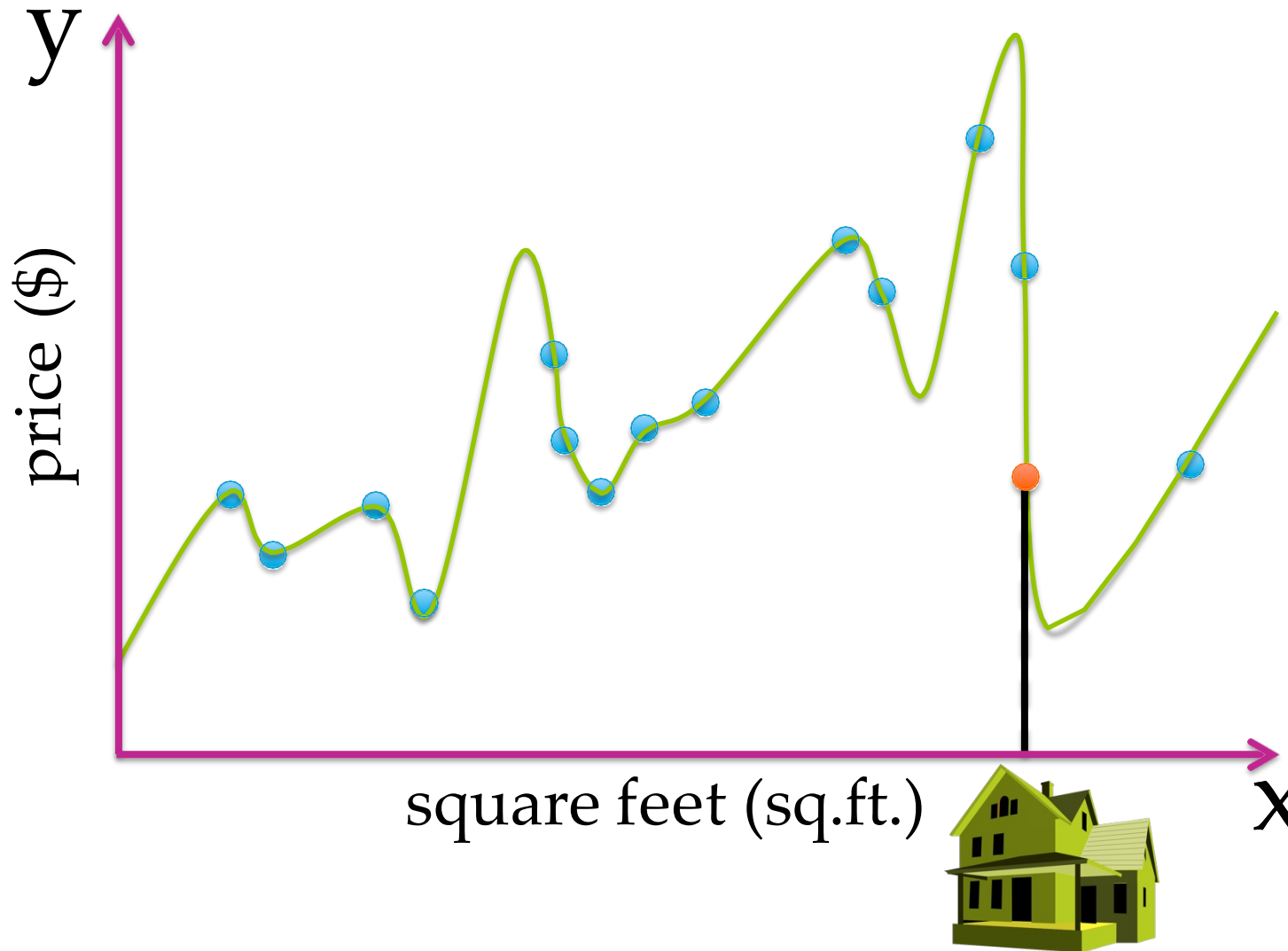
– test set



The Overfitting Problem in Regression

Predicting house prices

The Overfitting Problem in Regression



This happens often when we include too many features in regressions!

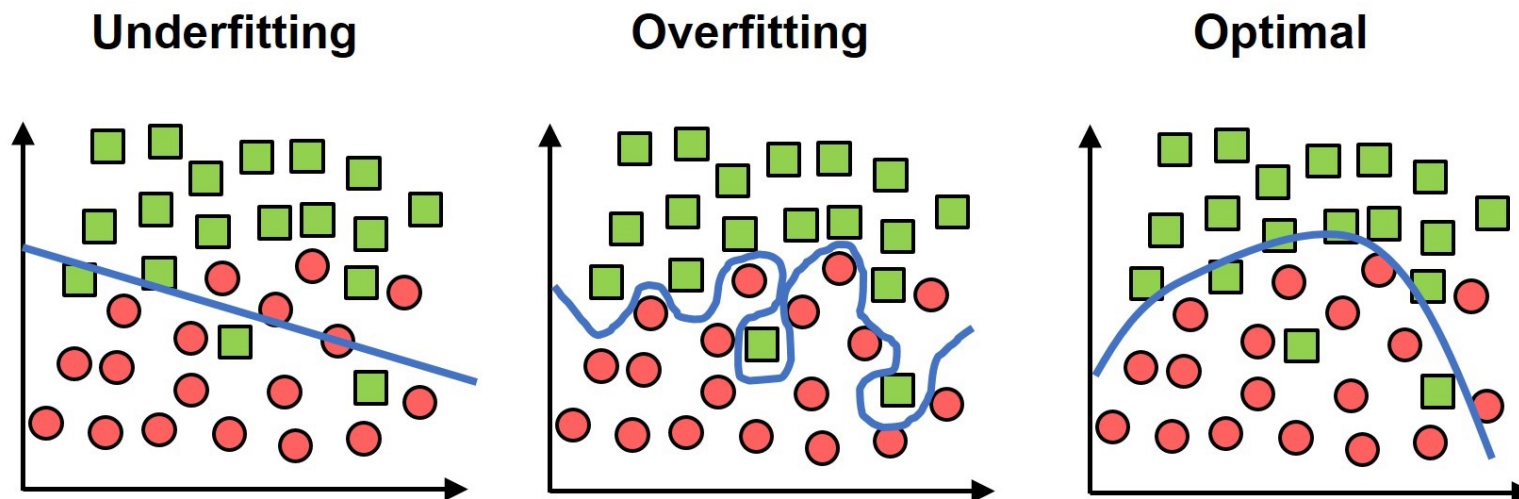
What else can we do?

Predicting house prices

The Overfitting Problem in Regression

Overfitting happens when a machine learning model learns the training data too well including the noise and random details. This makes the model to perform poorly on new, unseen data because it memorizes the training data instead of understanding the general patterns.

For example, if we only study last week's weather to predict tomorrow's i.e our model might focus on one-time events like a sudden rainstorm which won't help for future predictions.



Predicting house prices

Regularization in Machine Learning

Regularization is an important technique in machine learning that helps to improve model accuracy by preventing overfitting which happens when a model learns the training data too well including noise and outliers and perform poor on new data. By **adding a penalty for complexity** it helps simpler models to perform better on new data.

Types of Regularization

- Lasso
- Ridge
- Elastic Net

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \cdot \text{Regularization Term}$$

where

- m - Number of Features
- n - Number of Examples
- y_i - Actual Target Value
- $\hat{y}_i$ - Predicted Target Value

RSS for regressions

penalty for complexity

Predicting house prices

Lasso Regression (L1 Regularization)

A regression model which uses the L1 Regularization technique is called **LASSO (Least Absolute Shrinkage and Selection Operator)** regression.

It adds the absolute value of magnitude of the coefficient as a penalty term to the loss function(L). This penalty can shrink some coefficients to zero which helps in selecting only the important features and ignoring the less important ones.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$



Lasso regression adds the L1 norm (the sum of absolute values of coefficients) as the regularization term

Predicting house prices

Ridge Regression (L2 Regularization)

A regression model that uses the L2 regularization technique is called Ridge regression. It adds the squared magnitude of the coefficient as a penalty term to the loss function(L).

It handles multicollinearity by shrinking the coefficients of correlated features instead of eliminating them.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$


Ridge regression adds the L2 norm (the sum of squared coefficients) as the regularization term

Predicting house prices

Elastic Net Regression (L1 + L2 Regularization)

Elastic Net Regression is a combination of both L1 as well as L2 regularization.

That shows that we add the absolute norm of the weights as well as the squared measure of the weights. With the help of an extra hyperparameter that controls the ratio of the L1 and L2 regularization.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \left((1 - \alpha) \sum_{i=1}^m |w_i| + \alpha \sum_{i=1}^m w_i^2 \right)$$



Elastic Net combines Ridge and Lasso regression by adding both L1 and L2 regularization terms to the loss function:

Predicting house prices _____

Comparison of Lasso, Ridge, and Elastic Net

Feature	Lasso Regression	Ridge Regression	Elastic Net Regression
Regularization Type	L1 (absolute sum)	L2 (squared sum)	L1 + L2 combined
Feature Selection	Yes (some coefficients become zero)	No (retains all features)	Yes (some coefficients become zero)
Best Use Case	Sparse high-dimensional data	Multicollinearity	High-dimensional data with multicollinearity
Number of Features Used	Some features dropped	All features retained	Some features dropped
Complexity	Simple	Simple	Relatively complex

Predicting house prices

How to Choose the Right Model?

1. Lasso Regression:
 - Use when there are irrelevant or redundant features.
 - When you want automatic feature selection.
2. Ridge Regression:
 - Use when multicollinearity exists among features.
 - When you want to regularize feature impact without removing any features.
3. Elastic Net Regression:
 - Use when both multicollinearity and irrelevant features exist.
 - When you need the combined benefits of Ridge and Lasso.

Predicting house prices

Choosing the Right Regularization Parameter λ ?

Cross-validation is the most common method for choosing λ . The idea is to evaluate the model's performance over different values of λ by splitting the data into training and validation sets.

Steps:

1. Define a range of potential λ values (e.g., [0.001,0.01,0.1,1,10,100][0.001, 0.01, 0.1, 1, 10, 100][0.001,0.01,0.1,1,10,100]).
2. For each λ :
 - Train the model on the training set.
 - Evaluate its performance on the validation set using an appropriate metric (e.g., MSE for regression, accuracy for classification).
3. Choose the λ that minimizes the validation error.

Python Implementation

Lasso Regression

```
from sklearn.linear_model import Lasso
```

```
model = Lasso(alpha=1.0) # alpha controls regularization strength
model.fit(X_train, y_train)
print("Coefficients:", model.coef_)
```

Simply replace `LinearRegression()` with `Lasso()`

Ridge Regression

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=1.0) # alpha controls regularization strength
model.fit(X_train, y_train)
print("Coefficients:", model.coef_)
```

Simply replace `LinearRegression()` with `Ridge()`

Elastic Net Regression

```
from sklearn.linear_model import ElasticNet
```

```
model = ElasticNet(alpha=1.0, l1_ratio=0.5) # l1_ratio controls L1 vs L2 balance
model.fit(X_train, y_train)
print("Coefficients:", model.coef_)
```

Simply replace `LinearRegression()` with `ElasticNet()`

Python Implementation

Cross-Validation Example using Ridge Regression

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

# Define a range of lambda values (alpha in scikit-learn)
param_grid = {'alpha': [0.001, 0.01, 0.1, 1, 10, 100]}

# Perform GridSearchCV to find the best lambda
ridge = Ridge()
grid_search = GridSearchCV(ridge, param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(X_train, y_train)

# Best lambda
best_lambda = grid_search.best_params_['alpha']
print("Best lambda:", best_lambda)

# Fit the model using the best lambda
best_model = grid_search.best_estimator_
```

New Challenges in Regression

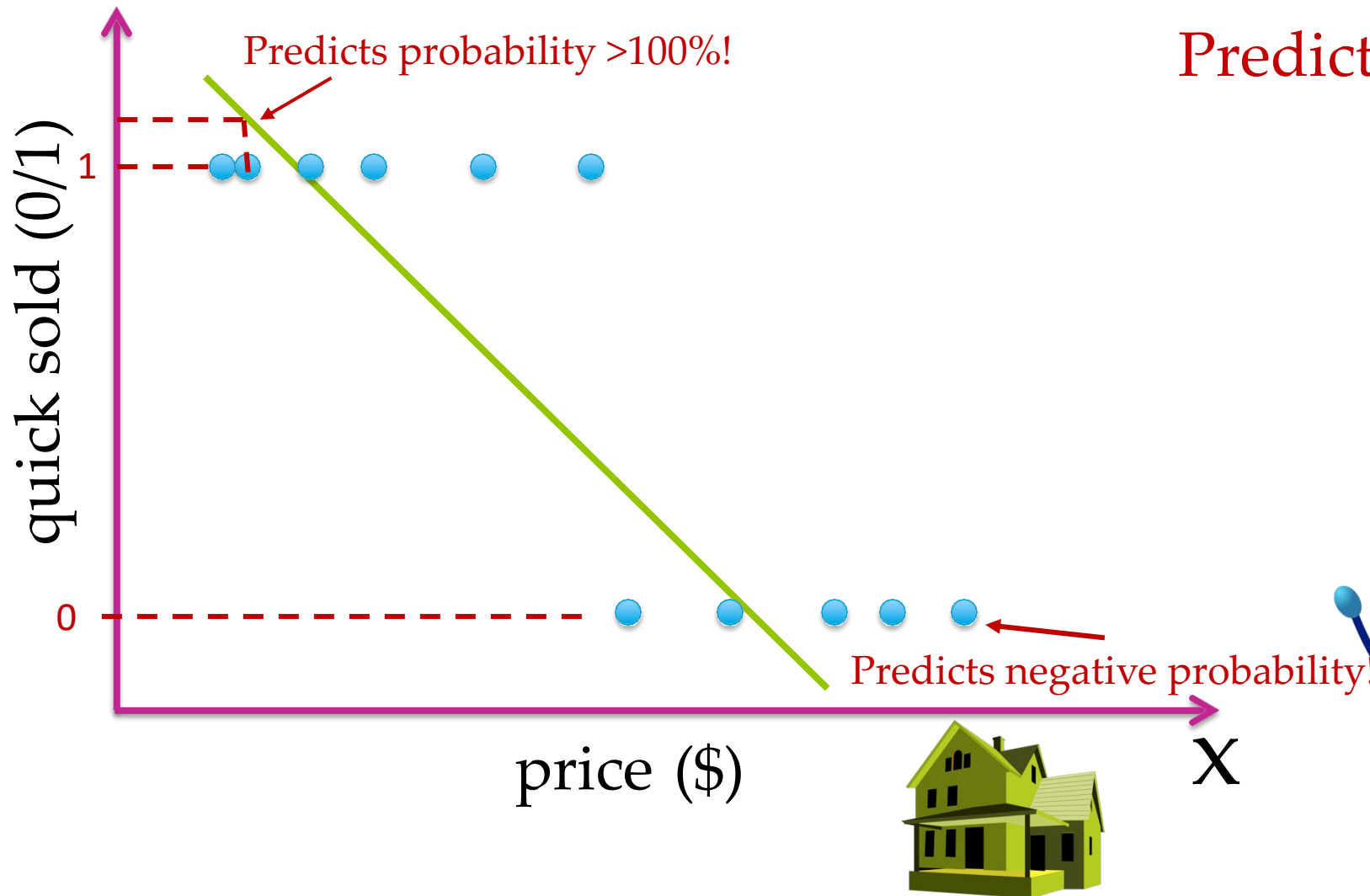
Predicting quick sold

Will my house sell quickly (< 30 days)? **Yes/No?**



Predicting quick sold

Problems with linear predictions for yes/no outcomes



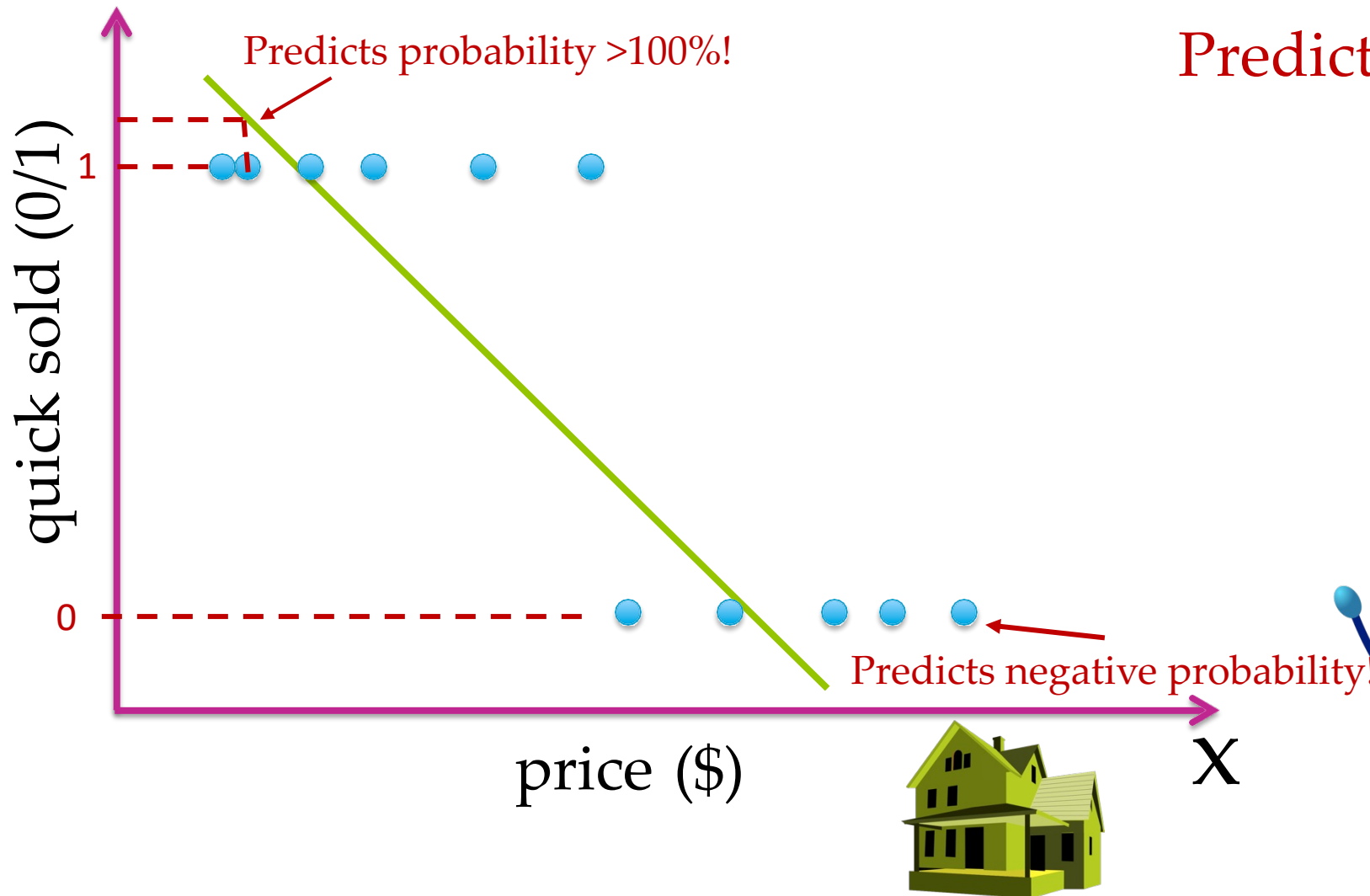
Predict for quick sold (0 or 1)

What does negative probability and probability > 100% mean?



Predicting quick sold

Problems with linear predictions for yes/no outcomes



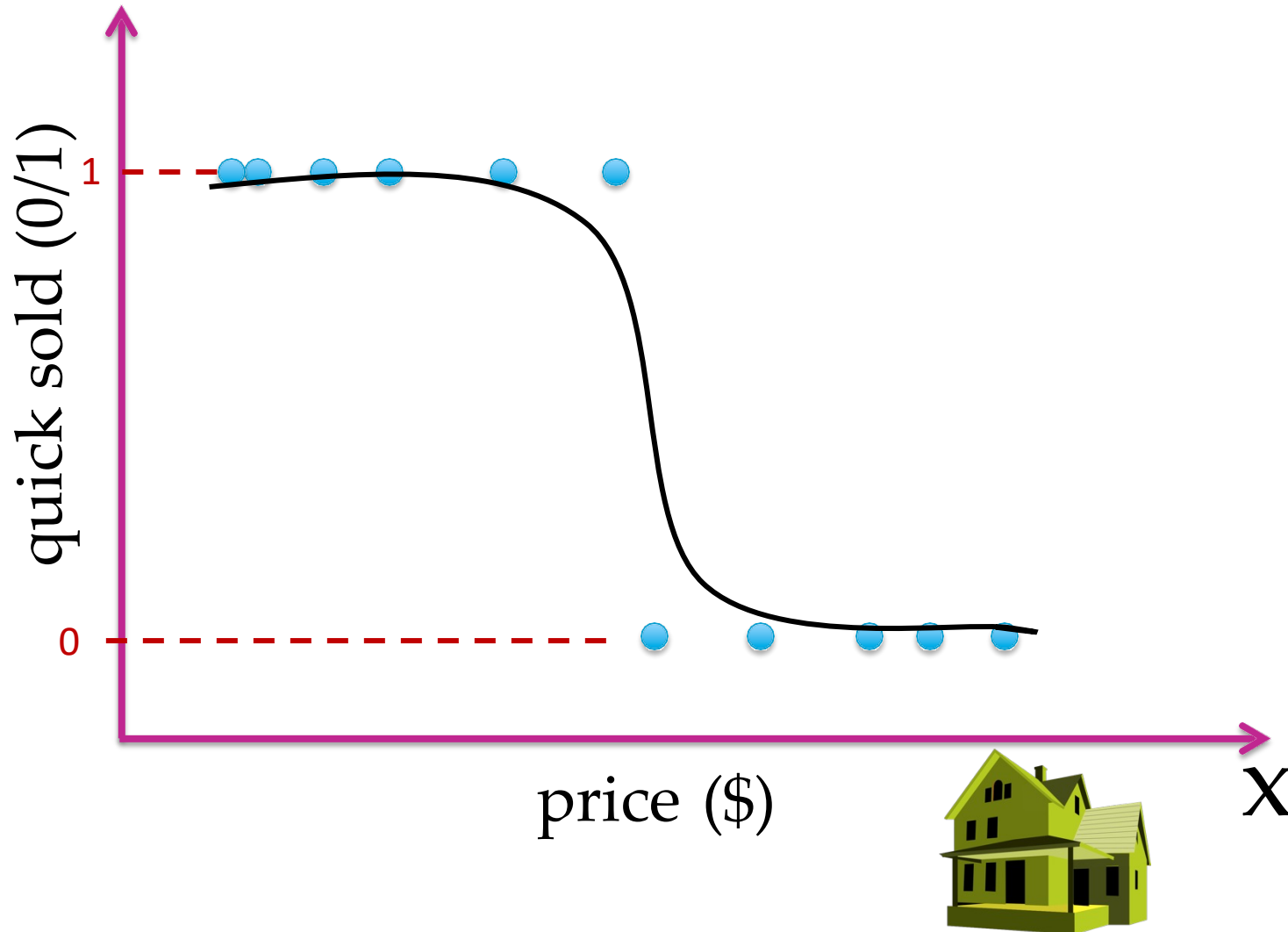
Predict for quick sold (0 or 1)

We need the predicted probability lie between 0 and 1

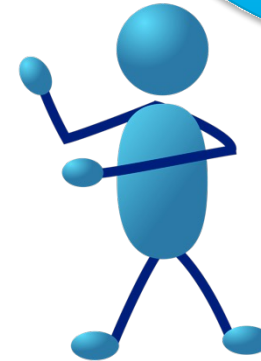


Predicting quick sold

Problems with linear predictions for yes/no outcomes

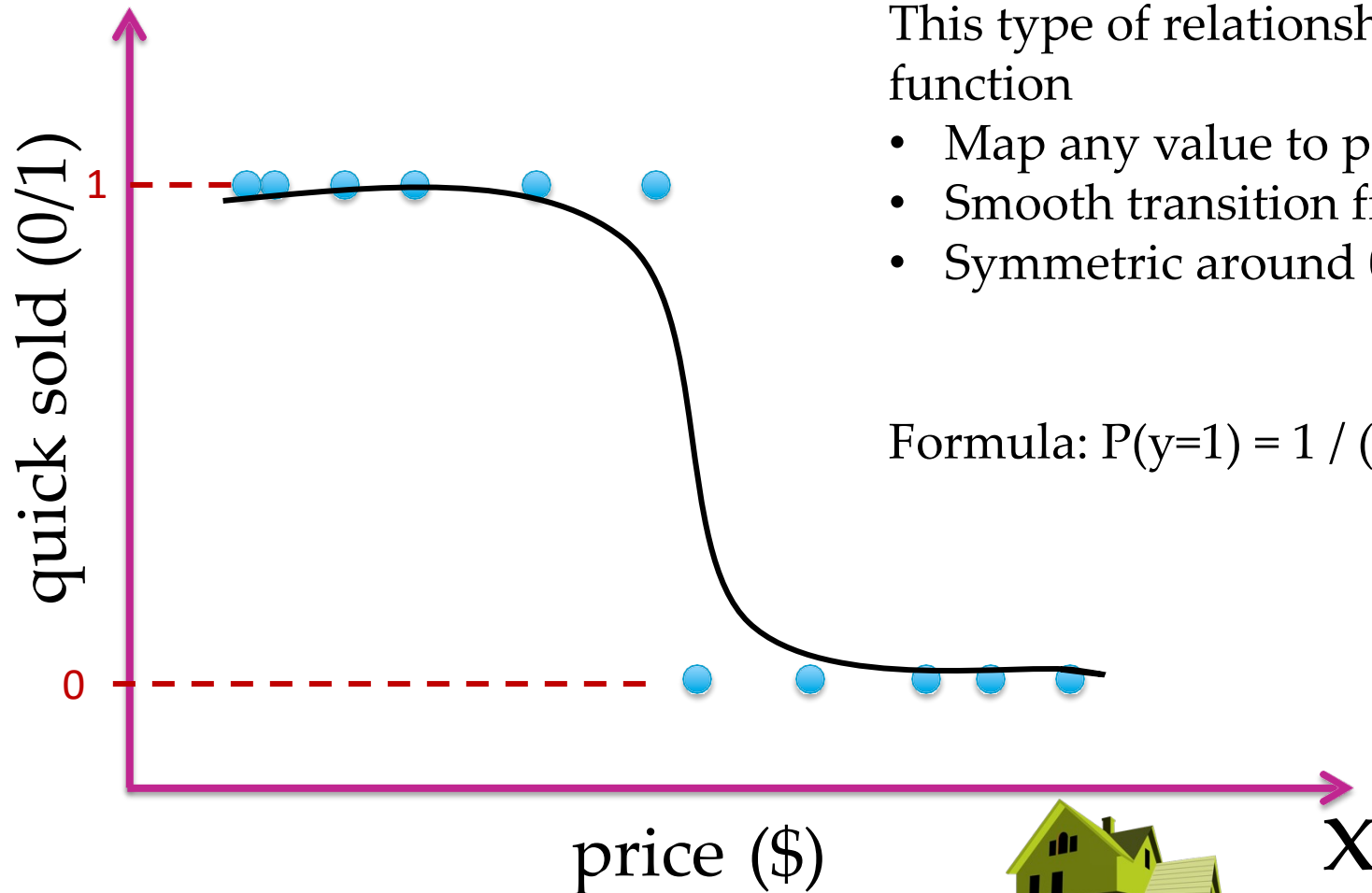


Here's a more realistic representation of the relationship



Predicting quick sold

Logistic Regression



This type of relationship is described by a special function

- Map any value to probability (0-1)
- Smooth transition from 0 to 1
- Symmetric around 0.5

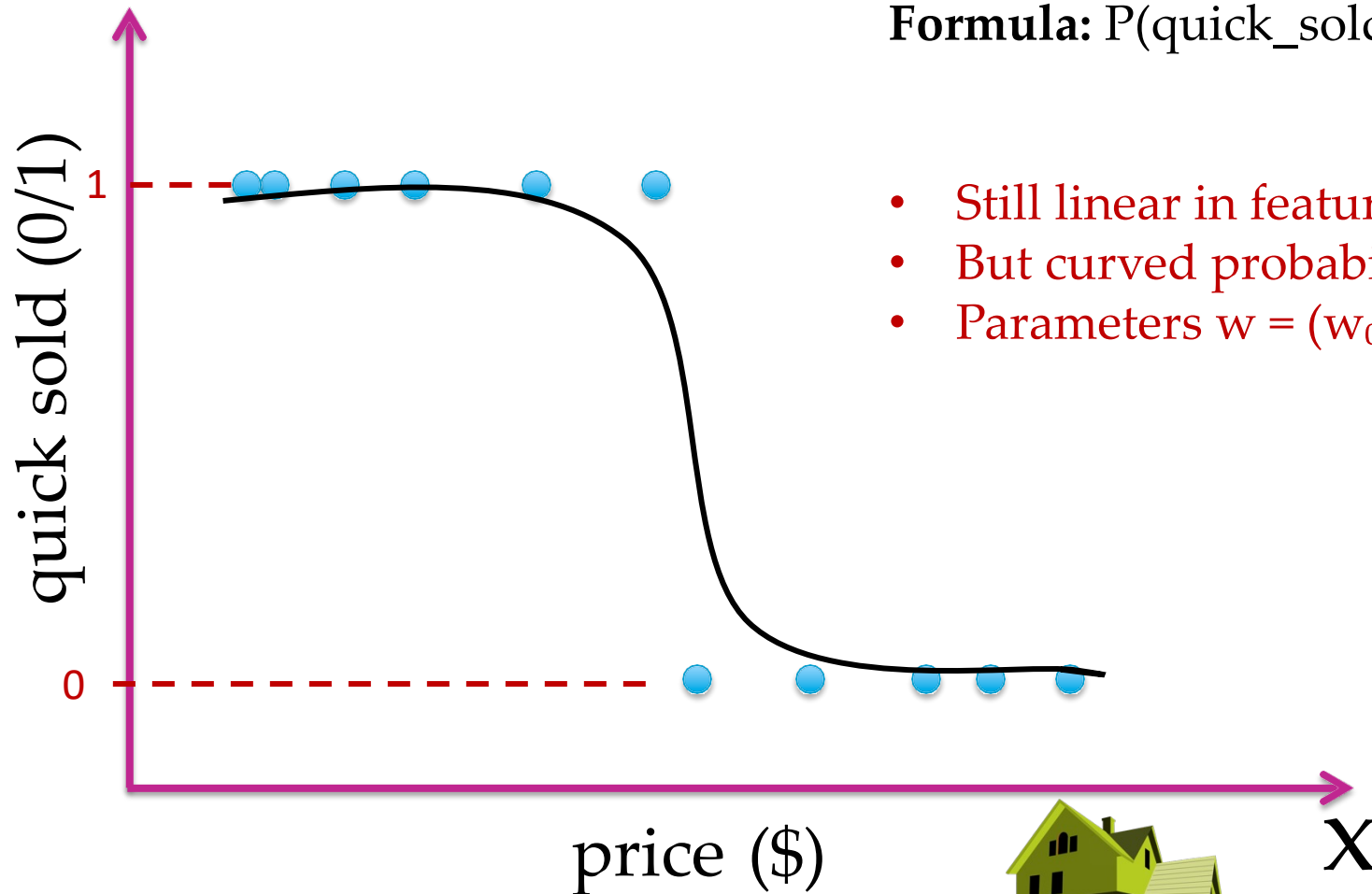
Formula: $P(y=1) = 1 / (1 + e^{(-f(x))})$



Predicting quick sold

Logistic Regression

Formula: $P(\text{quick_sold}) = 1 / (1 + e^{-(w_0 + w_1 \times \text{price})})$



- Still linear in features ($w_0 + w_1 \times \text{price}$)
- But curved probability output
- Parameters $w = (w_0, w_1)$ learned from data



Making Predictions with Logistic Regression

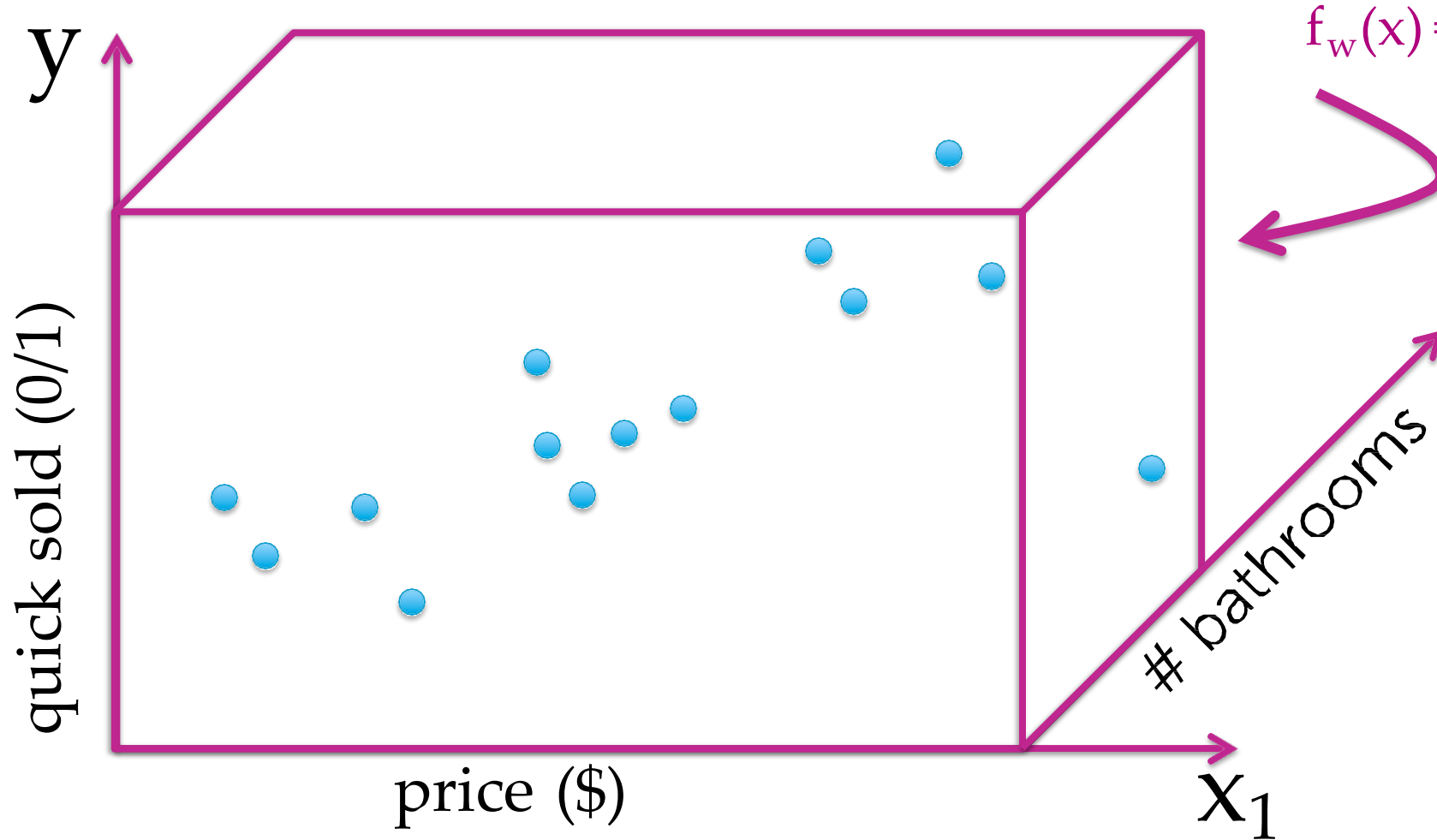


- Can adjust threshold: 0.3 (more liberal) vs 0.7 (more conservative)



Predicting quick sold

Multiple Features in Logistic Regression



$$f_w(x) = \frac{1}{1 + e^{-(w_0 + w_1 \text{price} + w_2 \# \text{bath})}}$$

- Each feature contributes to final probability
- Positive $w \rightarrow$ increases probability
- Negative $w \rightarrow$ decreases probability

Predicting quick sold

Training Logistic Regression

1. Cannot use RSS (residual sum of squares)
 - This doesn't really make sense here (remember the errors won't be normally distributed as there's only two values).
2. We use something called maximum likelihood to find w that maximizes probability of observed data
 - Maximum likelihood is an iterative process (gradient descent) that estimates the best fitted equation.
 - The iterative bit just means that we try lots of models until we get to a situation where tweaking the equation any further doesn't improve the fit.
 - The maximum likelihood bit is kind of complicated, although the underlying assumptions are simple to understand, and very intuitive. The basic idea is that we find the coefficient value that makes the observed data most likely.

Predicting quick sold

Evaluating Classification Models

Confusion Matrix

		Predicted	
		0	1
Actual	0	TN	FP
	1	FN	TP

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

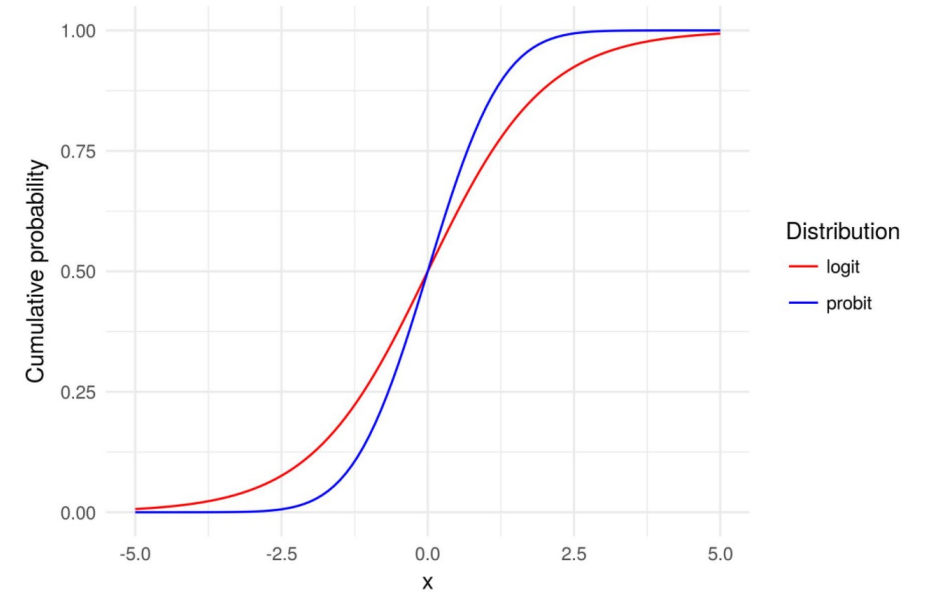
	slow_sold	quick_sold
slow_sold	180	20
quick_sold	15	85

- Precision = $85/(85+20) = 81\%$
- Recall = $85/(85+15) = 85\%$

Predicting quick sold

Logistic vs Probit Regression

- Two Common Classification Regression Methods
 - Logistic Regression: Uses Logit link function
 - Probit Regression: Uses Probit link function
- Mathematical Formula Comparison
 - Logistic: $P(y=1) = 1 / (1 + e^{(-f(x))}) = \text{Logit}^{(-1)}(f(x))$
 - Probit: $P(y=1) = \Phi(f(x))$ [Standard normal cumulative distribution function]
- Practical Application Differences
 - Similarities: Prediction results are usually very close
 - Logistic Advantages: Simpler computation, closed-form solution, easier interpretation
 - Probit Advantages: More rigorous theoretical foundation (based on normal distribution)
 - Selection Advice: Differences are minimal in practice, Logistic more commonly used



Python Implementation

Python Implementation

Create a simple logistic model

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# Load and prepare data
data = pd.read_csv("house_data.csv")
my_features = ['price'] # Define the feature column(s)

# Split data into training and testing sets
train_data, test_data = train_test_split(data, test_size=0.2, random_state=0)

# Separate features and target variable
X_train = train_data[my_features]
y_train = train_data['quick_sold'] # Target variable
X_test = test_data[my_features]
y_test = test_data['quick_sold']

# Train a logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)
```

Simply replace LinearRegression() with LogisticRegression()

Python Implementation

Model Evaluation

```
from sklearn.metrics import accuracy_score, classification_report

# Evaluate model
y_pred = model.predict(X_test) # Predictions on the test set
accuracy = accuracy_score(y_test, y_pred) # Calculate accuracy
print(f"Accuracy: {accuracy:.2f}")
```

```
# Detailed classification report (optional)
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
>>Accuracy: 0.36
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.36	1.00	0.53	1568
1	0.00	0.00	0.00	2755
accuracy			0.36	4323
macro avg	0.18	0.50	0.27	4323
weighted avg	0.13	0.36	0.19	4323

Python Implementation

Prediction

```
# Predict for a single house
house = X_test.iloc[0:1] # Select a single house (first row of test data)
house_probability = model.predict_proba(house)[0][1] # Probability of class
house_prediction = model.predict(house)[0] # Prediction for the single house
house_prediction = model.predict(house)[0] # Prediction for the single house
print(f"Probability: {house_probability:.2f}")
print(f"Prediction: {house_prediction}")
```

```
>>Probability: 0.49
Prediction: 0
```

```
# Check coefficients
print("Coefficients (weights for each feature):", model.coef_)
```

```
>>Coefficients (weights for each feature): [[-9.26357715e-08]] higher price→ lower probability
```

Tutorial

